



# Analyzing Data Sparsity in Global and Compact Support Radial Basis Functions for 3D Unstructured Mesh Deformation

Rabab Alomairy<sup>1(✉)</sup>, Qinglei Cao<sup>2</sup>, Hatem Ltaief<sup>3</sup>, and David Keyes<sup>3</sup>

<sup>1</sup> Computer Science and Artificial Intelligence Laboratory, Massachusetts Institute of Technology, Cambridge, USA

[rabab.alomairy@mit.edu](mailto:rabab.alomairy@mit.edu)

<sup>2</sup> Department of Computer Science, Saint Louis University, Missouri, USA

[qinglei.cao@slu.edu](mailto:qinglei.cao@slu.edu)

<sup>3</sup> Extreme Computing Research Center, Applied Mathematics and Computational Sciences Program, King Abdullah University of Science and Technology, KSA, Thuwal, Saudi Arabia

[{hatem.ltaief,david.keyes}@kaust.edu.sa](mailto:{hatem.ltaief,david.keyes}@kaust.edu.sa)

**Abstract.** In this work, we explore the data sparsity characteristics of different Radial Basis Function (RBF) kernels in the context of 3D unstructured mesh deformation. While RBF interpolation is a powerful method for generating high-quality adaptive meshes, solving the resulting boundary problems leads to large, dense linear systems that are computationally expensive and memory-intensive due to their cubic computational complexity and quadratic memory requirement. To address these challenges, we exploit the rank structure of the matrix operators by employing a Tile Low-Rank (TLR) Cholesky-based solver, which approximates off-diagonal matrix tiles up to an application-specific accuracy threshold. Our study compares global support RBFs and compact support RBFs, focusing on their effects on rank distribution and numerical accuracy. Using realistic 3D geometries of SARS-CoV-2 viruses from the Protein Data Bank (PDB), we evaluate various RBF kernels, analyze the corresponding matrix rank structures, and assess the backward error resulting from low-rank approximations for different kernel types. We conduct experiments on various shared-memory systems, demonstrating the performance scalability on massively parallel architectures. Leveraging the Hierarchical Computations on Manycore Architectures (HiCMA) library and the PaRSEC runtime system, we show how data sparsity accelerates large-scale mesh adaptation, providing valuable insights into the balance between computational efficiency and numerical accuracy.

**Keywords:** High-performance computing · Task-based runtime · Tile Low-Rank · Radial Basis Function · mesh deformation · Cholesky solver, PaRSEC

# 1 Introduction

Meshing the deformable contour of moving 3D bodies presents a significant computational challenge in fluid-structure interaction (FSI) applications. As the mesh evolves to track physical deformations, its quality can degrade over successive time steps, potentially leading to numerical instability or invalidation of the simulation. To preserve mesh integrity, mesh deformation can be used to adapt the mesh while maintaining its quality, preventing excessive distortion and numerical instability. Radial Basis Function (RBF) interpolation is widely used for its ability to generate high-fidelity unstructured meshes [18]. However, solving the associated dense linear system remains computationally expensive, requiring  $O(N^3)$  operations for  $N$  mesh points. To address this, Tile Low-Rank (TLR) Cholesky factorization is employed to exploit the inherent rank structure of RBF matrices, reducing computational and memory costs while maintaining numerical accuracy.

Our previous work primarily focused on studying the effect of hydrodynamics on a population of SARS-CoV-2 viruses by using Wendland RBF interpolation [4, 5]. This work focuses on analyzing the data sparsity properties of global and compact support RBF kernels. We compare global support functions, such as Gaussian and inverse multiquadratic, with compact support functions, including Wendland and Continuous Thin Plate Splines (CTPS). By leveraging Hilbert space-filling curve ordering, we enhance data locality and expose low-rank structures within the RBF matrices, further optimizing compression and computation.

As a realistic test case, we examine the effect of hydrodynamics on a population of SARS-CoV-2 viruses, leveraging newly available realistic 3D geometries. Our results indicate that among global support RBF kernels, Gaussian is the fastest, as its weights decay rapidly with distance, leading to lower-rank structures that optimize TLR solver. For compact support kernels, Wendland performs best, as we select a support radius that captures interactions between all mesh nodes, ensuring both computational efficiency and accuracy. We implement our approach using HiCMA, a high-performance hierarchical matrix computation library, and ParSEC, a task-based runtime system for efficient execution on distributed heterogeneous architectures. We evaluate our methodology across various hardware platforms, including Intel, AMD, and ARM, analyzing performance scaling, memory efficiency, and solver accuracy at different truncation thresholds.

To the best of our knowledge, this is the first study to systematically analyze both global and compact support RBF kernels using low-rank approximations. Through extensive performance evaluations, we examine the singular value decay of various RBF kernels. Furthermore, we demonstrate scalability on various shared-memory systems. Our findings reinforce the practical benefits of TLR-based solvers for large-scale 3D mesh deformation, offering insights for developing efficient RBF-based numerical solvers in HPC applications.

# 2 Related Work

RBF-based mesh deformation has been widely applied in aeronautics, control surface deflection, wing flapping, shape design, and more recently in hemody-

namics and other fields requiring mesh morphing. Its popularity stems from its robustness and ability to produce high-quality deformed meshes, particularly for unstructured grids. However, the method is computationally intensive, especially for large-scale problems with many nodes,  $N$ . Early work [19] reported that global RBF interpolation leads to dense, ill-conditioned matrices. Due to their zero-diagonal structure, iterative solvers require preconditioning to be effective. Compact support RBFs [18] were introduced to address these challenges, producing sparse systems and improving efficiency, though at the cost of reduced interpolation accuracy.

Both global and compact support RBF methods have seen significant advancements. Some studies have focused on parallelizing RBF computations [20], while others have explored data reduction techniques to mitigate computational complexity [29]. A common approach involves using a greedy algorithm to select a subset of boundary nodes, significantly reducing computational cost. However, this approach compromises geometric precision, particularly in regions with large deformations, ultimately limiting the method's effectiveness. Some other research efforts focus on accelerating dense linear algebra in RBF solvers. Work in [26] applies the Fast Multipole Method [21] to speed up RBF operator construction and iterative solvers. The effectiveness of preconditioning is crucial for improving convergence and computational efficiency. However, iterative solvers may be suboptimal for systems with multiple right-hand sides.

In this work, we employ a data compression approach to accelerate RBF solvers. Unlike data reduction methods, this approach enhances dense linear algebra operations by sparsifying the RBF system through low-rank compression based on a predefined threshold [4]. Low-rank approximations, particularly hierarchical matrices ( $\mathcal{H}$ -matrices) [22], have significantly reduced computational complexity and memory footprint in direct dense factorizations and solvers. Various state-of-the-art data compression formats exist for  $\mathcal{H}$ -matrix approximation, supporting weak admissibility (e.g., Hierarchically Semi-Separable (HSS) [31] and Hierarchically Off-Diagonal Low-Rank (HODLR) [9]) and strong admissibility (e.g.,  $\mathcal{H}^2$ -matrix [11] and Block/Tile Low-Rank (BLR/TLR) [7, 15]).

These approximation methods present challenges in 3D problems. Weak admissibility methods require high ranks for accuracy in large off-diagonal blocks, while strong admissibility methods face scalability limitations on distributed-memory systems. To address these issues, TLR formats have been used in 3D applications such as acoustic wave scattering and geospatial statistics by flattening recursion and minimizing synchronization overhead [2, 3]. Previous work [6] has explored RBFs for 2D interpolation using HODLR/HSS on a single core, with a parallel implementation [8] optimized for shared-memory systems via `OpenMP`. In this work, we evaluate the TLR method for both global and compact support RBFs on shared-memory architectures.

### 3 Radial Basis Function Interpolations

Radial Basis Functions (RBFs) are widely used for high-dimensional scattered data approximation, machine learning, solving high-order partial dif-

ferential equations (PDEs), and mesh deformation in fluid-structure interactions. A common mathematical foundation for these applications is multivariate interpolation, where a continuous function  $s(x)$  is approximated as:  $s(x) = \sum_{i=1}^{n_b} \alpha_i \phi(\|x - x_{b_i}\|) + p(x)$ , where  $\phi(r)$  is a radial basis function,  $x_{b_i}$  are known boundary nodes,  $n_b$  the number of boundary nodes, and  $p(x)$  is a low-order polynomial. The polynomial ensures linear consistency and surface continuity, maintaining rotational and translational invariance [10].

In mesh deformation, RBF interpolation derives the displacement of internal volume nodes from boundary nodes using a direct interpolation approach. The coefficients  $\alpha_i$  and  $\beta_l$  are computed by solving:

$$\begin{bmatrix} M_{b,b} & P_b \\ P_b^T & 0 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} d_b \\ 0 \end{bmatrix}, \quad (1)$$

where  $M_{b,b}$  is  $n_b \times n_b$  matrix a dense kernel matrix with entries  $\phi(\|x_{b_i} - x_{b_j}\|)$  and  $P_b$  contains polynomial terms.

For large-scale 3D deformations, solving this system is computationally challenging, as direct solvers scale as  $\mathcal{O}(n_b^3)$ . Iterative solvers struggle with high condition numbers and multiple right-hand sides. To address this, low-rank approximations are employed to reduce complexity.

Various RBF kernels exist [14, 18, 30, 32], broadly categorized into global and compact support functions. This study compares their effects on data sparsity and rank structure when using a Tile Low-Rank (TLR) Cholesky-based solver, evaluating their efficiency and numerical accuracy. Table 1 contains the list of RBF kernels that is considered in this study.

### Global Support RBFs.

account for interactions between all pairs of nodes, meaning their influence is always nonzero throughout the domain. While this results in higher accuracy, it also leads to dense system matrices,

making computations expensive. These functions are scaled by a shape parameter  $\delta$ , which controls the spread of influence and significantly affects both the accuracy and convergence of the RBF approximation.

The Gaussian RBF, introduced in geostatistics by Krige [25], is widely used for The exponential kernel, a variant of the Gaussian with a linear exponent, exhibits similar decay properties. The multiquadric RBF, introduced by Hardy [24], offers a broader influence than the Gaussian, assigning greater weight to distant points. While its quadratic and multiquadric forms are conditionally negative definite, the inverse variants are positive definite. In this work, we focus on the positive definite class due to its numerical stability.

**Table 1.** Radial basis function.

| Kernel                       | Abbrv. | Type    | $\mathbf{f}(\mathbf{r})$ |
|------------------------------|--------|---------|--------------------------|
| Gaussian                     | Gauss  | Global  | $\exp(-r^2)$             |
| Exponential                  | Exp    | Global  | $\exp(-r)$               |
| Inverse Quadratic            | IQ     | Global  | $\frac{1}{1+r^2}$        |
| Inverse Multi-Quadratic      | IMQ    | Global  | $\sqrt{\frac{1}{1+r^2}}$ |
| Continuous Thin Plate Spline | CPTS   | Compact | $(1-r)^5$                |
| Wendland                     | Wend   | Compact | $(1-r)^4(4r+1)$          |

**Compact Support RBFs** are defined as following:

$$\phi(r) = \begin{cases} f(r), & \text{if } 0 \leq r \leq 1 \\ 0, & \text{if } r > 1 \end{cases}$$

The support is controlled by the radius  $\delta$ , with the scaled distance given by  $r = ||x - x_{b_i}||/\delta$ . Thus, only mesh points within a sphere (3D) or a circle (2D) of radius  $\delta$  are influenced, while interactions beyond this range are ignored. Smaller  $\delta$  values yield sparser matrices, improving efficiency, whereas larger values enhance accuracy but result in denser system matrices. In this work, we consider Wendland function and the continuous variant of Thin Plate Spline (CTPS) as compact support RBFs. For all experiments, we select a large support radius to ensure that the function spans the entire space, producing a dense interpolation matrix.

## 4 Rank Structure Computations

We solve the RBF-based 3D unstructured mesh deformation problem using the Tile Low-Rank (TLR) Cholesky solver, as implemented in the HiCMA library [3, 28]. The TLR algorithm approximates off-diagonal tiles up to an application-defined accuracy threshold, reducing computational complexity while maintaining numerical fidelity. HiCMA integrates STARS-H routines and the  $\mathcal{H}$ -matrix market [27] to generate RBF kernel-based tiles and compress them on-the-fly using Randomized Singular Value Decomposition (RSVD). Two compression modes are supported: fixed rank, which applies uniform truncation across all tiles for performance-oriented preconditioning, and fixed accuracy, which dynamically adjusts tile approximation based on application-specific precision requirements. We adopt the fixed accuracy mode to ensure deformation accuracy.

HiCMA leverages BLAS/LAPACK and  $\mathcal{H}$ -core [1] libraries for low-rank computations, which are represented as a Directed Acyclic Graph (DAG), where nodes correspond to computational tasks and edges define data dependencies. Task execution is managed by dynamic runtime systems such as StarPU, PaRSEC, and OpenMP, which optimize scheduling across shared- and distributed-memory architectures while mitigating load imbalance. In this work, we employ PaRSEC to efficiently orchestrate task execution and maximize hardware utilization.

## 5 The PaRSEC Dynamic Runtime System

PaRSEC [12, 13] is a task-based runtime system designed for efficient task execution on shared- and distributed-memory heterogeneous architectures. It orchestrates task scheduling, resource allocation, and data movement, addressing key scalability challenges such as starvation, latency, overhead, and heterogeneity. PaRSEC provides several domain-specific languages (DSLs), including a customized compiler for executing Parameterized Task Graphs (PTG) which leverage Job Data Flow (JDF) to define task dependencies. It has demonstrated excellent performance in DPLASMA, a high-performance dense linear algebra library,

and has been optimized for low-rank computations [17, 28]. Recent enhancements improve load balancing, reduce memory usage, and minimize critical path execution time. To address the rank disparity in Tile Low-Rank (TLR) computations, ParSEC employs a hybrid data distribution combining band and 2D block cyclic layouts, reducing communication overhead by restricting data transfers to the actual tile rank. It further optimizes execution by dynamically trimming the DAG when all singular values in an off-diagonal tile fall below a given compression threshold, effectively reducing computational workload. Additionally, a diamond-shaped data distribution balances off-band tile computations, improving parallel efficiency [16]. In this work, we leverage these optimizations to conduct the analysis.

## 6 Construction of 3D Unstructured Mesh of the SARS-CoV-2 Virus

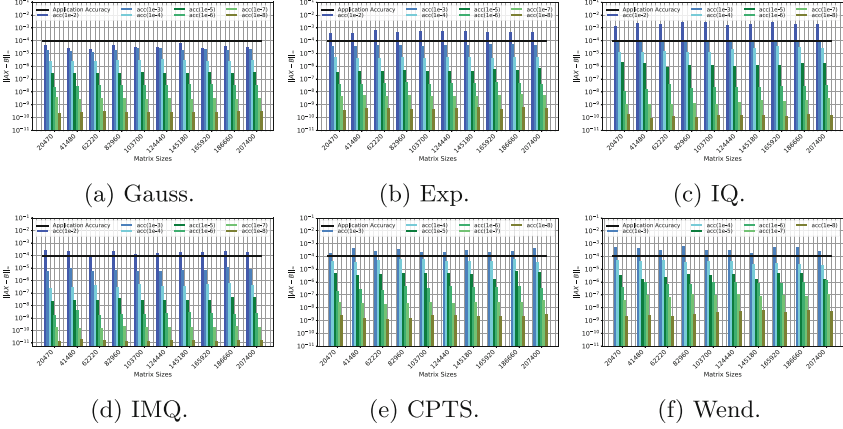
The multiscale procedure for constructing a single SARS-CoV-2 virus and generating a uniformly distributed viral population was detailed in our previous work [4]. Here, we adopt a different approach for confining the virus population using a sphere-packing strategy. Specifically, we construct a lattice of equal-radius spheres to fill a three-dimensional cubic space, with each sphere randomly occupied by a virus of the same radius. This virus-per-sphere filling strategy allows for a theoretical maximum packing density of approximately 74% in a cubic lattice [23]. To achieve a target density—i.e., maximizing the number of viruses within a droplet—we arrange spheres as closely as possible while ensuring no overlap. In this study, we vary the viral density within a fixed cubic domain of length  $1.7\mu m$  and analyze its impact on the resulting compressed RBF matrix.

## 7 Numerical Accuracy

In this section, we analyze the impact of accuracy thresholds and singular value decay across RBF kernels. We employ Hilbert ordering, which significantly enhances data locality, optimizes compression, and reduces computational overhead, as demonstrated in our previous study [4].

### 7.1 Accuracy of the TLR Cholesky Solver Using RBF Kernels

We evaluate the numerical accuracy of the TLR Cholesky solver for different RBF kernels by computing the residual, defined as  $\|Ax - b\|_\infty$ , where  $A$  is the RBF matrix,  $b$  the right-hand side, and  $x$  the computed solution. The residual is evaluated after deforming up to 20 SARS-CoV-2 viruses, each with a resolution of 10,370, as shown in Fig. 1. Lower truncation thresholds improve accuracy by increasing local ranks and reducing approximation error. However, if the residual falls below the required application accuracy (depicted as the horizontal line),

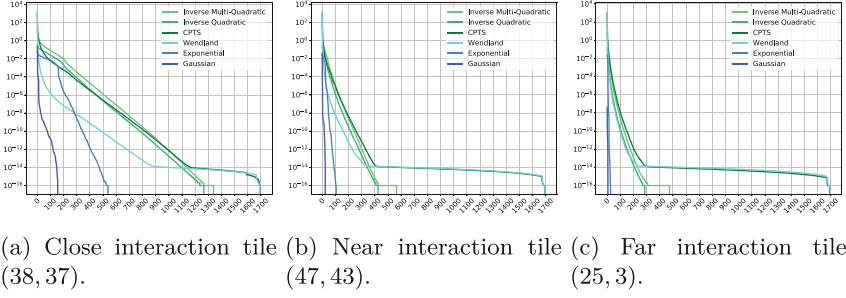


**Fig. 1.** Numerical accuracy of the TLR Cholesky solver using a population up to 20 SARS-CoV-2 viruses of size 10370 mesh points

unnecessary computations occur. The Gaussian kernel achieves the target accuracy with a truncation threshold of  $1e^{-2}$ , while exponential, inverse quadratic, and inverse multiquadratic kernels require  $1e^{-3}$ . Compact support functions, including CPTS and Wendland, reach application precision with  $1e^{-4}$ .

## 7.2 Singular Value Decay Across RBF Kernels After Compression

Figure 2 illustrates the singular value decay after compression using 10 viruses with a mesh resolution of 10370 and a tile size of 1700. The mesh is pre-processed with Hilbert ordering to enhance tile low-rank structure. Tile location is chosen based on distance from the diagonal, where higher ranks are concentrated. We analyze three regions: **close interaction**, where mesh points  $(i, j)$  are very close; **near interaction**, where they are moderately close; and **far interaction**, where they are significantly separated. The results show that as the interaction distance increases, singular values decay significantly across all RBF methods. Gaussian and exponential kernels exhibit the fastest decay due to their localized influence, whereas inverse quadratic and inverse multiquadratic kernels decay more gradually due to their global domain of influence. In contrast, compact support functions retain a relatively larger number of singular values, as their influence is constrained by the chosen support radius.



**Fig. 2.** Singular values decay using 10 viruses of size 10370 with tile size 1700.

## 8 Performance Evaluation

### 8.1 Environment Settings

The experiments are conducted on four shared-memory systems spanning Intel, AMD, and ARM architectures. The Intel-based systems include Cascade Lake (Xeon 6248) and Skylake (Xeon 6148), each equipped with two sockets, with 20 cores per socket. The AMD system features an EPYC Rome (7702) processor, configured with two sockets, each housing 64 cores. The ARM platform consists of a Cavium ThunderX2 system, also with two sockets, each containing 32 cores. These diverse platforms allow us to evaluate the performance portability and scalability of our TLR-based Cholesky solver across different memory hierarchies and processor designs. To maximize bandwidth and expose node parallelism on AMD systems, threads are evenly distributed across four-core complexes (CCX), each sharing an L3 cache. Depending on problem size and task count, we use 32 threads (one per CCX) or 64 threads (two per CCX) to balance bandwidth saturation and concurrency. Increasing thread count beyond this degrades performance due to bandwidth contention. Core binding is enforced via ParSEC’s thread placement mechanism using ParSEC’s environment variable for optimal bandwidth utilization.

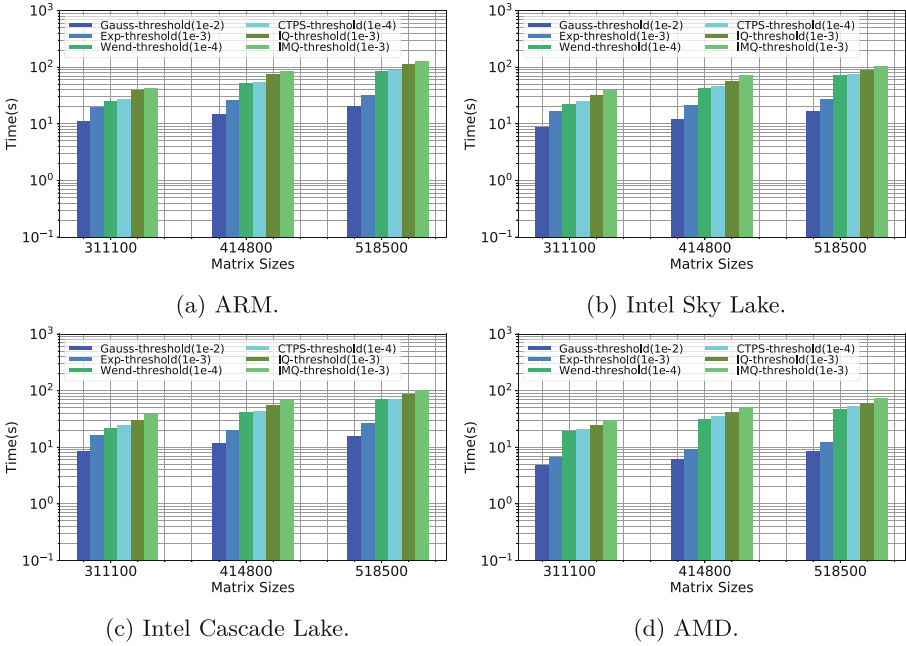
### 8.2 Performance in Shared-Memory System

**Comparing RBF Kernels:** Figure 3 presents the performance of the TLR Cholesky solver across RBF kernels on AMD, Intel, and ARM architectures. Gaussian achieves the highest performance due to its local influence and smaller ranks, while Wendland outperforms CPTS among compact support kernels.

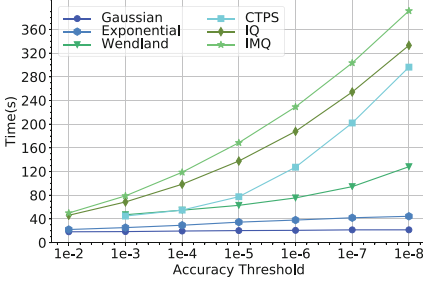
**Comparing Accuracy Thresholds:** Figure 4 shows the performance of the TLR Cholesky solver across varying truncation thresholds for different kernels. Accuracy thresholds have minimal impact on Gaussian and a slight effect on exponential. However, inverse quadratic and inverse multiquadratic exhibit significant performance variation. Among compact support functions, CPTS shows a larger performance gap than Wendland when adjusting thresholds.



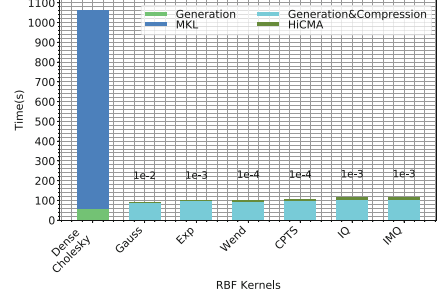
**Time Breakdown:** Figure 5 presents the time required for matrix generation, compression, and TLR Cholesky computation across all RBF kernels (each satisfying application accuracy requirements), compared to Intel MKL’s optimized dense Cholesky. The compression phase is computationally inexpensive due to its embarrassingly parallel nature. Once HiCMA is deployed, matrix generation becomes the dominant cost. TLR Cholesky computation is minimal, as Hilbert ordering effectively exploits tile low-rankness. Additionally, compression and computation time vary with rank distribution—kernels with overall lower ranks require less time than those with denser rank distributions. Finally, dense Cholesky computation remains independent of the chosen interpolation kernel.



**Fig. 3.** Performance of the TLR Cholesky solver using a population of viruses of size 10370.



**Fig. 4.** Accuracy threshold impact on the performance of TLR Cholesky solver on AMD Rome using 50 viruses of size 10370.



**Fig. 5.** Time breakdown of MKL and HiCMA on the AMD using 20 viruses of 10370 mesh points.

## 9 Conclusion

This work examines the impact of data sparsity in global and compact support RBF kernels for 3D unstructured mesh deformation, utilizing TLR Cholesky solver to reduce computational complexity while maintaining accuracy. Our analysis reveals distinct rank distribution behaviors across different RBF kernels. Gaussian and exponential exhibit low-rank characteristics due to their localized influence, whereas inverse quadratic and inverse multiquadratic maintain higher ranks as they decay more slowly over distance. Among compact support functions, Wendland achieves a better balance between accuracy and efficiency compared to CTPS, which requires lower truncation thresholds to reach the same accuracy. Performance evaluations indicate that Gaussian consistently delivers the best computational efficiency, while Wendland provides a well-balanced trade-off between accuracy and performance. These insights contribute to the development of more efficient RBF-based solvers in HPC applications, particularly for problems requiring scalable unstructured mesh adaptation.

**Acknowledgment.** For computer time, this research used the Ibex cluster and Shaheen-II supercomputer at the Supercomputing Laboratory of the King Abdullah University of Science and Technology (KAUST) in Thuwal, Saudi Arabia. This research was supported by the Defense Advanced Research Projects Agency (DARPA HR00112490488), and the internal awards from Saint Louis University (Grant-0001651 and PROJ-000498). In addition, Authors would like to acknowledge KAUST Ibn Rushd post-doctoral fellowship which supported Rabab Alomairy.

## References

1. Akbudak, K., Alomairy, R., Ltaief, H., Keyes, D.: Hcore (2017). <https://github.com/ecrc/hcore>
2. Akbudak, K., Ltaief, H., Mikhalev, A., Charara, A., Esposito, A., Keyes, D.: Exploiting data sparsity for large-scale matrix computations. In: Aldinucci, M., Padovani, L., Torquati, M. (eds.) Euro-Par 2018. LNCS, vol. 11014, pp. 721–734. Springer, Cham (2018). [https://doi.org/10.1007/978-3-319-96983-1\\_51](https://doi.org/10.1007/978-3-319-96983-1_51)
3. Al-Harthi, N., et al.: solving acoustic boundary integral equations using high performance tile low-rank LU factorization. In: Sadayappan, P., Chamberlain, B.L., Juckeland, G., Ltaief, H. (eds.) ISC High Performance 2020. LNCS, vol. 12151, pp. 209–229. Springer, Cham (2020). [https://doi.org/10.1007/978-3-030-50743-5\\_11](https://doi.org/10.1007/978-3-030-50743-5_11)
4. Alomairy, R., Bader, W., Ltaief, H., Mesri, Y., Keyes, D.: High-performance 3D unstructured mesh deformation using rank structured matrix computations. ACM Trans. Parallel Comput. **9**(1), 1–23 (2022)
5. Alomairy, R.M.: High-Performance Scientific Applications Using Mixed Precision and Low-Rank Approximation Powered by Task-based Runtime Systems. PhD thesis, King Abdullah University of Science and Technology (2022)
6. Ambikasaran, S., Darve, E.: An  $O(N \log N)$  fast direct solver for partial hierarchically semi-separable matrices. J. Sci. Comput. **57**(3), 477–501 (2013)
7. Amestoy, P., Ashcraft, C., Boiteau, O., Buttari, A., L’Excellent, J.-Y., Weisbecker, C.: Improving multifrontal methods by means of block low-rank representations. SIAM J. Sci. Comput. **37**(3), A1451–A1474 (2015)
8. Aminfar, A.: A Fast Direct Solver for HODLR Matrices (2014). <https://github.com/amiraa127/DenseHODLR>
9. Aminfar, A., Ambikasaran, S., Darve, E.: A fast block low-rank dense solver with applications to finite-element matrices. J. Comput. Phys. **304**, 170–188 (2016)
10. Beckert, A., Wendland, H.: Multivariate interpolation for fluid-structure-interaction problems using radial basis functions. Aerosp. Sci. Technol. **5**(2), 125–134 (2001)
11. Börm, S.: Efficient Numerical Methods for Non-Local Perators: H2-Matrix Compression, Algorithms and Analysis, vol. 14. European Mathematical Society (2010)
12. Bosilca, G., Bouteiller, A., Danalis, A., Faverge, M., Hérault, T., Dongarra, J.J.: PaRSEC: exploiting Heterogeneity to Enhance Scalability. Comput. Sci. Eng. **15**(6), 36–45 (2013)
13. Bouteiller, A., Hérault, T., Cao, Q., Schuchart, J., Bosilca, G.: PaRSEC: scalability, flexibility, and hybrid architecture support for task-based applications in ECP. Inter. J. High Performance Comput. Appli. (2024)
14. Buhmann, M.D.: Radial Basis Functions: Theory and Implementations, vol. 12. Cambridge University Press (2003)
15. Cao, Q., et al.: Reshaping geostatistical modeling and prediction for extreme-scale environmental applications. In: SC22: International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–12. IEEE (2022)
16. Cao, Q., et al.: A framework to exploit data sparsity in tile low-rank cholesky factorization. In: 2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 414–424. IEEE (2022)
17. Cao, Q., et al.: Leveraging parsec runtime support to tackle challenging 3d data-sparse matrix problems. In: 2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 79–89. IEEE (2021)

18. De Boer, A., Van der Schoot, M.S., Bijl, H.: Mesh deformation based on radial basis function interpolation. *Comput. Struct.* **85**(11), 784–795 (2007). Fourth MIT Conference on Computational Fluid and Solid Mechanics
19. Dyn, N., Levin, D., Rippa, S.: Numerical procedures for surface fitting of scattered data by radial functions. *SIAM J. Sci. Stat. Comput.* **7**(2), 639–659 (1986)
20. Estruch, O., Lehmkuhl, O., Borrell, R., Pérez Segarra, C.D., Oliva, A.: A parallel radial basis function interpolation method for unstructured dynamic meshes. *Comput. Fluids* **80**, 44–54 (2013)
21. Greengard, L., Rokhlin, V.: A fast algorithm for particle simulations. *J. Comput. Phys.* **73**(2), 325–348 (1987)
22. Hackbusch, W.: A sparse matrix arithmetic based on *mathcal{H}*-Matrices part I: introduction to *mathcal{H}*-Matrices. *Computing* **62**(2), 89–108 (1999)
23. Hales, T., et al.: A Formal Proof of the Kepler Conjecture. In: *Forum of Mathematics, Pi*, vol. 5. Cambridge University Press (2017)
24. Hardy, R.L.: Multiquadric equations of topography and other irregular surfaces. *J. Geophys. Res.* **76**(8), 1905–1915 (1971)
25. Krige, D.G.: A statistical approach to some basic mine valuation problems on the witwatersrand. *J. Southern African Instit. Mining Metallurgy* **52**(6), 119–139 (1951)
26. Le Borne, S., Wende, M.: Multilevel interpolation of scattered data using  $\mathcal{H}$ -matrices. *Numerical Algorithms* **40**(4), 1497–1526 (2020)
27. Mikhalev, A., Alomairy, R., Ltaief, H., Keyes, D.: STARS-H (2017). <https://github.com/ecrc/stars-h>
28. Qinglei, C., et al.: Extreme-scale task-based cholesky factorization toward climate and weather prediction applications. In: *Proceedings of the Platform for Advanced Scientific Computing Conference*. ACM (2020)
29. Rendall, T.C.S., Allen, C.B.: Parallel efficient mesh motion using radial basis functions with application to multi-bladed rotors. *Int. J. Numer. Meth. Eng.* **81**(1), 89–105 (2010)
30. Rendall, T.C.S., Allen, C.B.: Unified fluid-structure interpolation and mesh motion using radial basis functions. *Intern. J. Numerical Methods Eng.* **74**(10), 1519–1559 (2008)
31. Rouet, F.-H., Li, X.S., Ghysels, P., Napov, A.: A distributed-memory package for dense hierarchically semi-separable matrix computations using randomization. *ACM Trans. Math. Softw. (TOMS)* **42**(4), 27 (2016)
32. Wendland, H.: Error estimates for interpolation by compactly supported radial basis functions of minimal degree. *J. Approx. Theory* **93**(2), 258–272 (1998)